

PROJECT REPORT



EXAM EASE

College Examination Management System

CSE2006 – Programming in Java

Student Name:- Ishan Jain

Registration Number:- 25BAI11017

Faculty / Supervisor:- Dr. Manorma Chauhan

1. INTRODUCTION

ExamEase is a console-based College Examination Management System that I built in Java for the CSE2006 Programming in Java course, under the "Build Your Own Project" category. The idea behind the project was to pick a problem that colleges actually face and try to solve a part of it using the object-oriented concepts we learned in class, without making the scope so big that it becomes unmanageable.

Most examination-related record keeping in a college involves three things: keeping track of students, keeping track of exams, and figuring out where each student will sit during an exam. Normally this is either done on paper or in scattered Excel sheets, which makes it easy to make mistakes like duplicate IDs, missing records, or seating two people in the same seat. ExamEase tries to bring these three tasks into one simple menu-driven Java application.

The project has three main modules:

Student Management – lets the admin add, view, search and delete student records, and prevents duplicate Student IDs from being entered.

Exam Management – lets the admin add, view, search and delete exam records (subject, date, time, duration), again with duplicate Exam ID detection.

Seating Allocation – takes an exam, checks how many seats are available, and if there are enough, automatically assigns sequential seat numbers (A01, A02, A03, ...) to the registered students and saves the final seating chart to a text file.

The application is entirely console-based (no GUI, no database) and runs through a simple numbered menu system. All data used in a session is stored in memory using ArrayLists, and the only persistent output the program produces is the seating allocation text file. This report documents the requirements, design, implementation and testing of the project as it currently stands.

2. PROBLEM STATEMENT

Managing examination information manually can become difficult when there are multiple students and examinations. Student records, examination details and seating arrangements may have to be maintained separately, which can make searching and organizing information more time-consuming.

Another common problem is preparing examination seating arrangements. The number of available seats needs to be checked before students are assigned seats. If there are fewer seats than students, the allocation should not proceed.

The aim of ExamEase is to provide a simple console-based application that can manage student and examination records and generate a basic examination seating arrangement.

The system is designed as a small-scale project and focuses only on the main examination management tasks rather than trying to implement a complete college examination system.

3. FUNCTIONAL REQUIREMENTS

The following are the main functional requirements of ExamEase:-

3.1 Student Management

The system should allow the administrator to:

- Add a new student.
- Enter the Student ID, name, program, semester and section.
- View all available students.
- Search for a student using Student ID.
- Delete a student using Student ID.
- Detect and reject duplicate Student IDs.

The Student ID is stored as a String because college registration numbers can contain both letters and numbers, for example:

25BAI11017

3.2 Exam Management

The system should allow the administrator to:

- Add a new examination.
- Enter the Exam ID, subject, date, time and duration.
- View all available examinations.
- Search for an examination using Exam ID.
- Delete an examination using Exam ID.
- Detect and reject duplicate Exam IDs.

3.3 Seating Allocation

The system should allow the administrator to:

- Select an existing examination.
- Enter the number of available seats.
- Compare the number of students with the available seats.
- Detect insufficient seating capacity.
- Assign seats sequentially to students.
- Display the generated seating arrangement.
- Save the seating arrangement into a text file.

For example:

Student ID	Seat Number
25BAI11016	A01
25BAI11017	A02
25BAI10537	A03

3.4 Exception Handling

The system uses custom exceptions for specific situations:

- `StudentNotFoundException`
- `ExamNotFoundException`
- `InsufficientSeatsException`

These exceptions help the program handle errors in a clear way instead of terminating unexpectedly.

3.5 File Generation

After successful seating allocation, the program creates a text file containing the examination information and the assigned seats.

For example:

1001_seating.txt

The file contains the Exam ID, subject, student IDs and their respective seat numbers.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Usability

The application uses a simple numbered console menu. The administrator can select the required operation by entering a number.

This makes the application easy to understand and operate without requiring a graphical interface.

4.2 Reliability

The system handles common errors such as:

- Duplicate Student IDs
- Duplicate Exam IDs
- Searching for records that do not exist
- Insufficient seats
- Invalid numerical input

This helps prevent incorrect operations.

4.3 Maintainability

The project is divided into different packages and classes according to their responsibilities.

For example:

- `model`
- `manager`
- `exception`
- `util`

This makes the code easier to understand and modify.

4.4 Performance

The current project is designed for a small number of student and examination records. `ArrayList` is used to store the records, which is sufficient for the current scope of the application.

4.5 Resource Efficiency

The application does not require a database or large external resources. Student and examination records are maintained during program execution, while the seating arrangement is stored in a small text file.

5.SYSTEM ARCHITECTURE

The main program is handled by `Main.java`, which displays the menus and takes input from the administrator.

The manager classes are responsible for operations related to students and examinations.

The model classes represent the different objects used by the application.

Custom exception classes are used to handle application-specific errors.

`FileManager` handles the creation of the seating output file.

5.1 Main Components

Main.java

Handles:

- Main menu
- User input
- Student menu
- Exam menu
- Seating allocation flow

Model Package

Contains:

- User
- Student
- Admin
- Exam
- Seat

These classes represent the main objects of the system.

Manager Package

Contains:

- StudentManager
- ExamManager

These classes handle operations on student and examination records.

Exception Package

Contains:

- `StudentNotFoundException`
- `ExamNotFoundException`
- `InsufficientSeatsException`

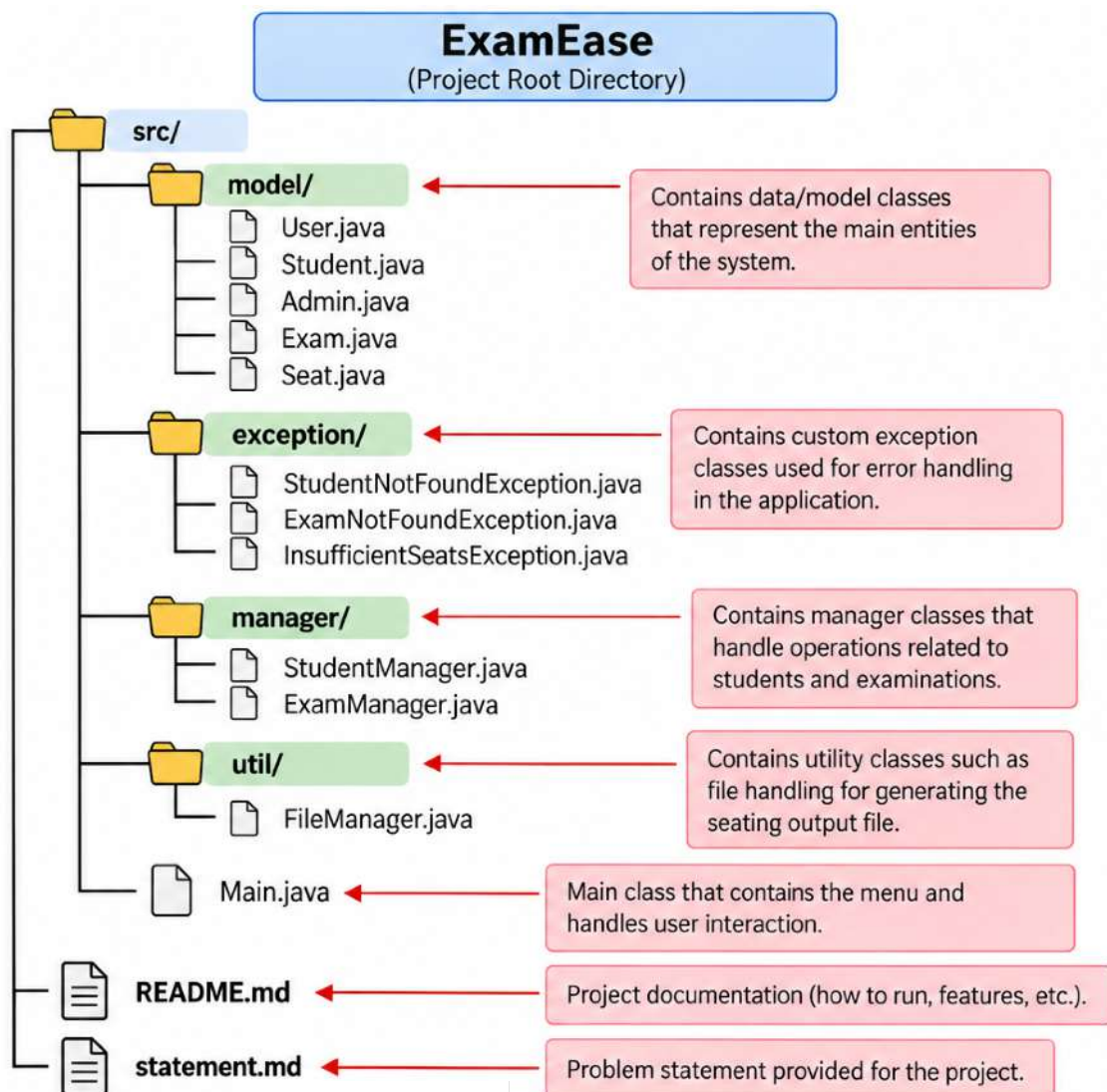
Utility Package

Contains:

- `FileManager`

It is responsible for saving seating information to a text file.

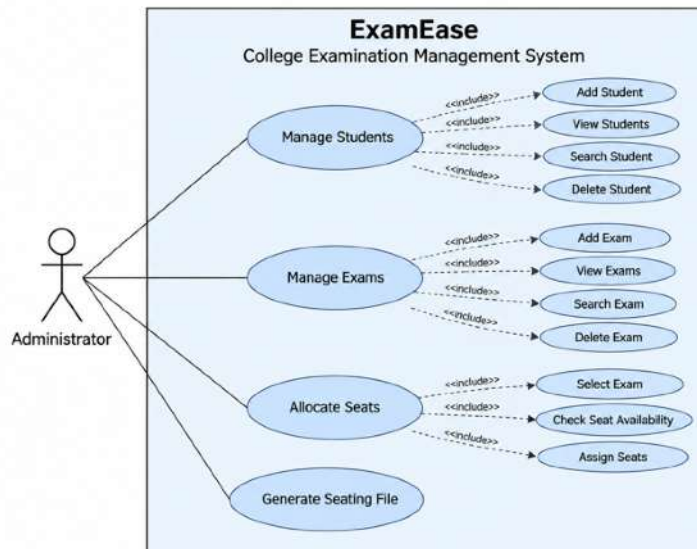
5.2 Project Structure



6. DESIGN DIAGRAMS

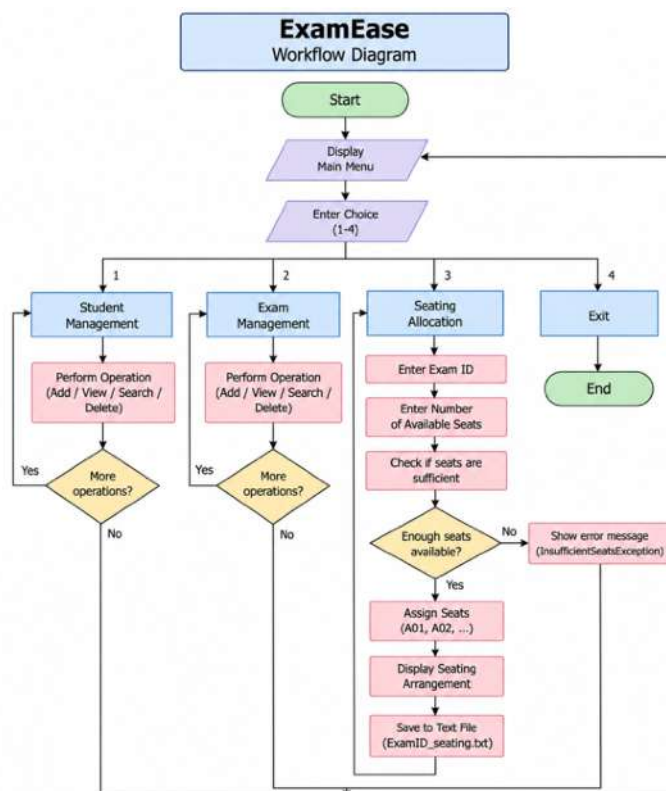
6.1 Use Case Diagram

The Use Case Diagram shows how the administrator interacts with the ExamEase system. The administrator can manage student records, manage examination records, allocate seats and generate the seating file.



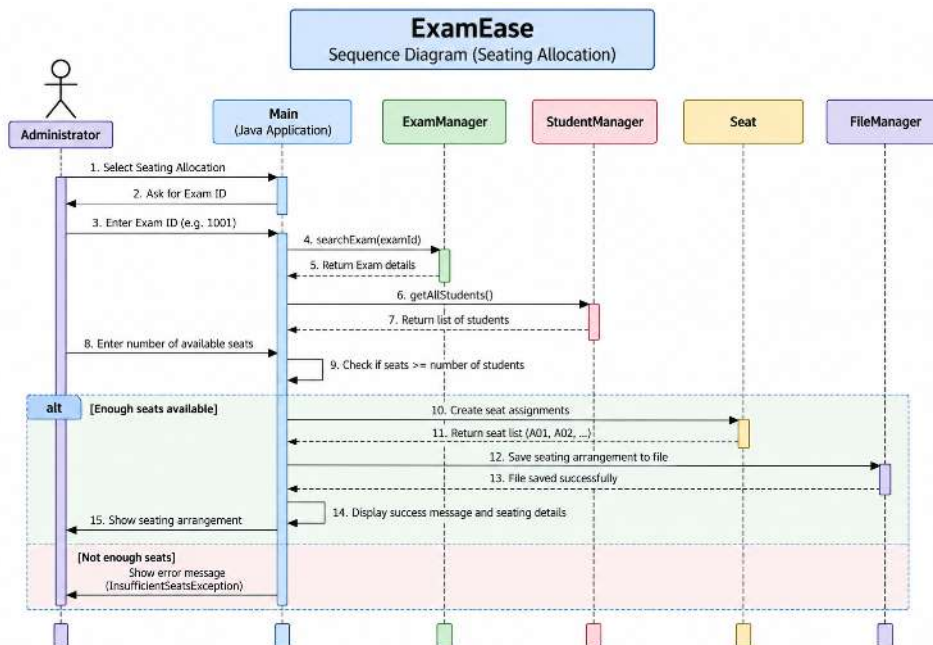
6.2 Workflow Diagram

The Workflow Diagram represents the flow of the ExamEase application from starting the program to selecting a module and performing the required operation. It also shows the seating capacity check before seat allocation.



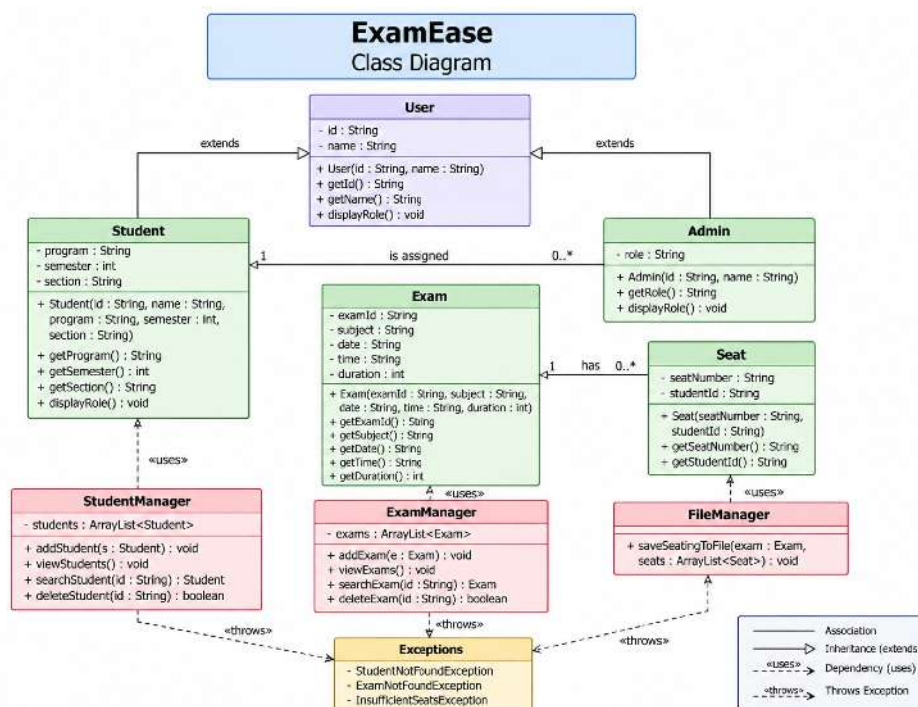
6.3 Sequence Diagram

The Sequence Diagram shows the interaction between the administrator and different components of the system during seating allocation. It represents how student and exam information is obtained, seats are assigned and the final seating arrangement is saved to a file



6.4 Class / Component Diagram

The Class Diagram shows the main classes used in ExamEase and their relationships. The `Student` and `Admin` classes inherit from the `User` class, while the manager classes handle student and examination operations. `FileManager` is responsible for generating the seating output file.



7. DESIGN DECISIONS & RATIONALE

7.1 Console-Based Interface

A console-based interface was selected because it keeps the application simple and easy to use. It also allows the main Java concepts used in the project to be demonstrated clearly without adding the extra complexity of a graphical interface.

7.2 Use of ArrayList

`ArrayList` is used to store student and examination records. It allows objects to be added, searched, displayed and removed easily. It also demonstrates the use of the Java Collections Framework.

7.3 Student ID as String

The Student ID is stored as a `String` because college registration numbers contain both letters and numbers.

For example:

25BAI11017

Using an integer would not be suitable for this type of ID.

7.4 Separate Packages

The project is divided into different packages according to their responsibilities.

- `model` – contains the main data classes.
- `manager` – handles student and examination operations.
- `exception` – contains custom exception classes.
- `util` – handles file generation.

This structure makes the project easier to understand and maintain.

7.5 Use of Custom Exceptions

Custom exceptions are used to handle situations specific to the application.

The project uses:

- `StudentNotFoundException`
- `ExamNotFoundException`

- `InsufficientSeatsException`

This makes error messages more meaningful and keeps exception handling organized.

7.6 Sequential Seat Allocation

The seating system assigns seats sequentially, starting from:

A01, A02, A03, A04...

This approach was selected because it is simple, easy to implement and suitable for the current scope of the project.

7.7 Text File Output

The seating arrangement is saved in a text file using Java file handling. A text file was chosen because the current project does not require a database and the generated seating information is small.

8. IMPLEMENTATION DETAILS

8.1 Project Structure

The ExamEase project is organized into different packages and classes so that each part of the application has a clear responsibility. The `model` package contains classes such as `Student`, `Exam`, `Seat`, `User` and `Admin` that represent the main objects of the system. The `manager` package contains `StudentManager` and `ExamManager`, which handle operations related to student and examination records. The `exception` package contains custom exceptions used to handle errors such as missing students, missing exams and insufficient seats. The `util` package contains `FileManager`, which is responsible for saving the seating arrangement to a text file. Finally, `Main.java` controls the menus, takes input from the administrator and connects the different parts of the application. This structure keeps the code organized and makes the project easier to understand and maintain.

8.2 Student Management

The `Student` class represents student information in the system.

The `StudentManager` class maintains an `ArrayList<Student>` containing the available student records.

The module provides the following operations:

- Add Student
- View Students
- Search Student
- Delete Student

Before adding a student, the program checks whether the Student ID already exists. This prevents duplicate student records.

If a student is searched or deleted using an ID that does not exist, the program handles the situation using `StudentNotFoundException`.

8.3 Exam Management

The `Exam` class represents examination information.

Each examination contains:

- Exam ID
- Subject
- Date
- Time
- Duration

The `ExamManager` class stores examination objects using an `ArrayList<Exam>`.

The module provides:

- Add Exam
- View Exams
- Search Exam
- Delete Exam

The program also checks for duplicate Exam IDs before adding a new examination.

8.4 Seating Allocation

The Seating Allocation module is used to assign seats to students for a particular examination.

The administrator first enters the Exam ID and the number of available seats.

The program then checks whether the number of available seats is sufficient for all students.

If sufficient seats are available, the students are assigned sequential seat numbers.

For example:

Student ID	Seat Number
25BAI11016	A01

25BAI11017	A02
25BAI10537	A03

If the number of available seats is less than the number of students, the program handles the situation using `InsufficientSeatsException`.

8.5 Object-Oriented Programming

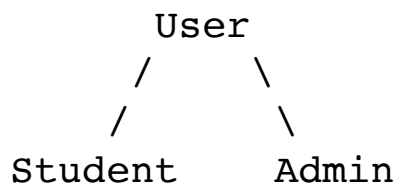
Object-Oriented Programming concepts are used throughout the project.

Encapsulation

The data members of classes such as `Student`, `Exam` and `Seat` are kept private. Getter methods are used to access the required information.

Inheritance

The `Student` and `Admin` classes inherit common properties and methods from the `User` class.



Abstraction

`User` is implemented as an abstract class containing common information and behaviour.

Polymorphism

The `displayRole()` method is overridden in the child classes to provide different behaviour.

Classes and Objects

The system represents different entities using Java classes such as:

- `User`
- `Student`

- Admin
- Exam
- Seat

8.6 Exception Handling

Exception handling is used to prevent the program from terminating unexpectedly when an error occurs.

The project uses custom exceptions for:

- Student not found
- Exam not found
- Insufficient seating capacity

The program also handles invalid numerical input entered through the console.

8.7 File Handling

Java file handling is used to save the seating arrangement.

The `FileManager` class uses `FileWriter` to generate the seating file.

For example, if the Exam ID is 1001, the generated file is:

1001_seating.txt

The file contains the examination information and the assigned seats.

Example:

Exam: 1001

Subject: MATHS

Student ID	Seat Number
25BAI11016	A01
25BAI11017	A02
25BAI10537	A03

9. SCREENSHOTS / RESULTS

The following screenshots were taken during the actual execution of the ExamEase application.

9.1 Main Menu

```
===== MAIN MENU =====  
1. Student Management  
2. Exam Management  
3. Seating Allocation  
4. Exit  
Enter choice: 
```

Figure 9.1: ExamEase Main Menu

The main menu provides access to the three major modules of the system:

1. Student Management
2. Exam Management
3. Seating Allocation

The fourth option is used to exit the application.

9.2 Student Management

```
===== STUDENT MANAGEMENT =====  
1. Add Student  
2. View Students  
3. Search Student  
4. Delete Student  
5. Back
```

Figure 9.2: Student Management Menu

The Student Management menu provides options for adding, viewing, searching and deleting student records.

9.3 Exam Management

```
===== EXAM MANAGEMENT =====
1. Add Exam
2. View Exams
3. Search Exam
4. Delete Exam
5. Back
```

Figure 9.3: Exam Management Menu

The Exam Management menu provides options for adding, viewing, searching and deleting examination records.

9.4 Seating Allocation

```
===== MAIN MENU =====
1. Student Management
2. Exam Management
3. Seating Allocation
4. Exit
Enter choice: 3

===== SEATING ALLOCATION =====
Enter Exam ID: █
```

Figure 9.4: Seating Allocation

The Seating Allocation module asks the administrator to enter an Exam ID and proceeds with the seating allocation process after checking the available capacity.

9.5 Generated Seating File

```
1 Exam: 1001
2 Subject: MATHS
3
4 Student ID  Seat Number
5 -----
6 25BAI11016  A01
7 25BAI11017  A02
8 25BAI10537  A03
9
```

Figure 9.5: Generated Seating File

The generated seating file for Exam 1001 contains the examination details and the seat numbers assigned to the students.

10. TESTING APPROACH

Testing was performed manually by running the application through the console. Both normal operations and possible error conditions were tested.

Test Case	Input / Situation	Expected Result	Status
T1	Add a valid student	Student is added successfully	PASS
T2	Add duplicate Student ID	Duplicate ID is rejected	PASS
T3	Search existing student	Student details are displayed	PASS
T4	Search non-existing student	StudentNotFoundException is handled	PASS
T5	Add a valid exam	Exam is added successfully	PASS
T6	Search non-existing exam	ExamNotFoundException is handled	PASS
T7	Allocate seats with sufficient capacity	Seats are assigned sequentially	PASS
T8	Allocate seats with insufficient capacity	InsufficientSeatsException is handled	PASS
T9	Enter invalid numerical input	User is asked to enter valid input	PASS
T10	Complete seating allocation	Seating text file is generated	PASS

Testing Result

The tested operations produced the expected results. The application was able to manage student and examination records, handle common errors, allocate seats and generate the seating output file.

11. CHALLENGES FACED

11.1 Handling Alphanumeric Student IDs

One of the main issues encountered during development was handling Student IDs correctly.

Initially, the Student ID was treated as a number. However, actual college registration numbers contain letters as well as numbers, such as:

25BAI11017

The Student ID was therefore changed to the `String` data type in the related classes.

11.2 Maintaining Consistency Between Classes

After changing the Student ID from an integer to a `String`, the change had to be made in several places, including constructors, getter methods, search methods and ID comparisons.

This helped in understanding how classes in a Java project are connected to each other.

11.3 Handling Scanner Input

The program uses `Scanner` for taking input from the user.

Mixing `String` and integer inputs can sometimes result in input-related problems. A separate method for reading integer values was used to make numerical input safer.

11.4 Exception Handling

Another challenge was deciding how the application should respond when a student or examination does not exist.

Custom exceptions were created so that different types of errors could be identified and handled separately.

11.5 Checking Seating Capacity

The program must ensure that there are enough seats before assigning seats to students.

A capacity check was therefore added before the allocation process. If the available seats are insufficient, the program displays an appropriate error instead of assigning incomplete seating arrangements.

11.6 File Generation

The seating arrangement had to be saved separately after successful allocation.

A `FileManager` class was used for this purpose so that file-writing operations remain separate from the main program and menu logic.

12. CONCLUSION

ExamEase is a simple Java-based College Examination Management System developed to manage student records, examination records and basic seating allocation.

The project successfully combines several Java programming concepts into one practical application. It uses Object-Oriented Programming, inheritance, abstraction, polymorphism, encapsulation, collections, exception handling and file handling.

The application allows the administrator to add, view, search and delete student and examination records. It also checks seating capacity, assigns seats sequentially and saves the final seating arrangement into a text file.

The project was intentionally kept small and focused so that the core Java concepts could be implemented and understood properly. Through this project, practical experience was gained in organizing Java code, handling user input, debugging errors and connecting different classes to perform a complete workflow.